

UD3 - Programación Orientada a Objetos en PHP

Alumno: Francisco Pérez Ruiz **Asignatura:** Desarrollo Web en Entorno Servidor (DWES)

Introducción

En esta unidad damos el salto al paradigma de la **programación orientada a objetos** en PHP. Hasta ahora todo lo que veníamos haciendo era código procedural (funciones sueltas, scripts cortos). Con POO, en cambio, organizamos el código en **clases** que tienen propiedades y métodos, y creamos **objetos** (instancias) que usan esas clases.

La POO se basa en cuatro pilares: **abstracción**, **encapsulamiento**, **herencia** y **polimorfismo**. A lo largo de los programas vamos pasando por todos ellos, además de tocar temas más avanzados como traits, interfaces, namespaces, autoloader y algunas estructuras de datos de la SPL.

Una ventaja real de trabajar así es separar la lógica de negocio de la presentación. Eso facilita el mantenimiento, la reutilización de código, la escalabilidad, las pruebas y el trabajo en equipo.

Programa 1 - Clase básica

Primer contacto con las clases. Defino una **ClaseSencilla** que tiene una propiedad **\$numero** con valor 10 y un método **mostrarVar()** que la imprime. Después creo un objeto con **new** y llamo al método con el operador **->**.

Programa1.php · líneas 1-13

```
1 <?php
2 class ClaseSencilla
3 {
4     public $numero = '10';
5
6     public function mostrarVar() {
7         echo ("Francisco va a sacar ") . $this->numero . " en PHP";
8     }
9 }
10
11 $obj = new ClaseSencilla();
12 $obj->mostrarVar();
13 ?>
```

Salida:

Francisco va a sacar 10 en PHP

Programa 2 - Instancias y constructores

Aquí ya hay dos clases en el mismo archivo: **Animal** y **Coche**. La idea es practicar el constructor **__construct**, que es el método que se ejecuta automáticamente cuando creas un objeto con **new**. Sirve para inicializar las propiedades.

Clase **Animal** con **\$especie** y **\$edad**:

```

1 <?php
2 class Animal {
3     // Propiedades de la clase
4     public $especie;
5     public $edad;
6
7     // Constructor de la clase
8     public function __construct($especie, $edad) {
9         $this->especie = $especie; // Asigna el valor del parámetro $nombre a la propiedad $especie del objeto
10        $this->edad = $edad;      // Asigna el valor del parámetro $edad a la propiedad $edad del objeto
11    }
12
13    // Método para mostrar información de la persona
14    public function mostrarInfo() {
15        return "Especie: $this->especie, Edad: $this->edad <br>";
16    }
17 }
18
19 // Crear una instancia de la clase Animal usando el constructor
20 $especie1 = new Animal("Perro", 8);
21 echo $especie1->mostrarInfo(); // Salida: Nombre: Perro, Edad: 8
22
23 $persona2 = new Animal("Gato", 5);
24 echo $persona2->mostrarInfo(); // Salida: Nombre: Gato, Edad: 5
25 ?>

```

Clase **Coche** con **\$marca** y **\$modelo**:

```

27 <?php
28 class Coche {
29     public $marca;
30     public $modelo;
31
32     public function __construct($marca, $modelo) {
33         $this->marca = $marca;
34         $this->modelo = $modelo;
35     }
36     // Método para obtener información del coche
37     public function obtenerInformacion() {
38         return "Este es un coche de la marca $this->marca, modelo $this->modelo. <br>";
39     }
40 }
41
42 // Crear instancias de la clase Coche
43 $coche1 = new Coche("Peugeot", "Corolla");
44 $coche2 = new Coche("Ford", "Focus");
45
46 echo $coche1->obtenerInformacion(); // Salida: "Este es un coche de la marca Toyota, modelo Corolla."
47 echo $coche2->obtenerInformacion(); // Salida: "Este es un coche de la marca Ford, modelo Focus."
48

```

Salida:

Especie: Perro, Edad: 8
Especie: Gato, Edad: 5
Este es un coche de la marca Peugeot, modelo Corolla.
Este es un coche de la marca Ford, modelo Focus.

Programa 3 - Constructor y destructor

Tres instancias de **Persona**. Importante: el método `__destruct` se ejecuta automáticamente cuando el objeto deja de existir (al final del script, normalmente). No hay que llamarlo a mano, PHP lo hace por nosotros.

```
1 <?php
2 class Persona
3 {
4     private $nombre;
5
6     public function __construct($nombre)
7     {
8         $this->nombre = $nombre;
9         echo "Objeto creado con nombre: $this->nombre <br>";
10    }
11
12    public function saludar()
13    {
14        echo "Hola, soy $this->nombre <br>";
15    }
16
17    public function __destruct()
18    {
19        echo "<br>Objeto destruido.";
20    }
21 }
22
23 //Crea al menos 3 objetos de la clase persona
24 // y muestra un saludo para cada uno
25
26 $persona1 = new Persona("Francisco");
27 $persona2 = new Persona("Lucía");
28 $persona3 = new Persona("Guerola");
29
30 $persona1->saludar();
31 $persona2->saludar();
32 $persona3->saludar();
33 ?>
```

Salida:

Objeto creado con nombre: Francisco
Objeto creado con nombre: Lucía
Objeto creado con nombre: Guerola
Hola, soy Francisco
Hola, soy Lucía
Hola, soy Guerola

Objeto destruido.
Objeto destruido.
Objeto destruido.

Programa 4 - Getters y setters

Ejemplo clásico de encapsulamiento: las propiedades `$mayor` y `$menor` son `private`, así que no se pueden tocar desde fuera. Para leerlas y escribirlas existen los métodos `getMayor`, `getMenor`, `setMayor` y `setMenor`. El operador `?` en el tipo de retorno indica que la función puede devolver `null`.

Clase `MayorMenor`:

Programa4.php · líneas 1-21

```
1 <?php
2 class MayorMenor {
3     private int $mayor;
4     private int $menor;
5
6     public function setMayor(int $may) {
7         $this->mayor = $may;
8     }
9
10    public function setMenor(int $men) {
11        $this->menor = $men;
12    }
13
14    public function getMayor() : int {
15        return $this->mayor;
16    }
17
18    public function getMenor() : int {
19        return $this->menor;
20    }
21 }
```

Función **maymen** que crea un objeto, le pasa los valores y lo devuelve:

Programa4.php · líneas 23-39

```
23 //crea una nueva función que devuelve un nuevo objeto con los valores mayor y menor que se le pasen, la ? indica que puede devolver null
24 function maymen(array $numeros) : ?MayorMenor {
25     $a = max($numeros);
26     $b = min($numeros);
27
28     $result = new MayorMenor();
29     //Establece el MAYOR y el MENOR
30     $result->setMayor($a);
31     $result->setMenor($b);
32     //devuelve el resultado
33     return $result;
34 }
35
36 $resultado = maymen([1,76,9,388,41,39,25,97,22]);
37 echo "<br>Mayor: ".$resultado->getMayor();
38 echo "<br>Menor: ".$resultado->getMenor();
39 ?>
```

Salida:

Programa 5 - `$this` y métodos estáticos

Aquí se ve la diferencia entre llamar a un método desde una instancia (con `$this` disponible) y desde un método estático (sin `$this`). La clase `C` tiene un método `static`, así que se invoca con `C::callTestThis()` sin necesidad de crear una instancia.

Clases A y B:


```

1  <?php
2  class A
3  {
4      function testThis()
5      {
6          if (isset($this)) {
7              echo '$this está definida (';
8              echo get_class($this);
9              echo ")\n";
10         } else {
11             echo "\$this no está definida.\n";
12         }
13     }
14 }
15
16 class B
17 {
18     function testB()
19     {
20         // A::testThis();
21         $a= new A();
22         $a->testThis();
23     }
24 }

```

Clase C estática y llamadas finales:

```

26 //Crea clase C con método estático que llame a testThis de A
27 class C {
28     public static function callTestThis() {
29         $a = new A();
30         $a->testThis();
31     }
32 }
33
34
35 $a = new A();
36 $a->testThis();
37
38 //A::testThis();
39
40 $b = new B();
41 $b->testB();
42
43 // B::bar();
44 C::callTestThis();
45 ?>

```

Salida:

\$this está definida (A) \$this está definida (A) \$this está definida (A)

Programa 6 - Propiedad y método con el mismo nombre

Las propiedades y los métodos viven en espacios separados, así que pueden compartir nombre. La forma de distinguirlos es el contexto: `$obj->bar` accede a la propiedad, `$obj->bar()` llama al método. Aquí los dos se llaman `bar`.

Programa6.php · líneas 1-12

```
1 <?php
2 class Foo
3 {
4     public $bar = 'property';
5
6     public function bar() {
7         return 'method';
8     }
9 }
10
11 $obj = new Foo();
12 echo $obj->bar, PHP_EOL, $obj->bar(), PHP_EOL;
```

Salida (separadas por `PHP_EOL`, que es un salto de línea):

Programa 7 - Encapsulamiento

`$saldo` es `private`, así que solo se puede modificar a través de `depositar()`. Si intentara acceder directamente desde fuera con `$cuenta->saldo`, daría error. Dentro de los métodos uso `$this->saldo`, donde `$this` se refiere al objeto actual.

```
1 <?php
2 class Cuenta {
3     private $saldo = 0;
4
5     public function depositar($cantidad) {
6         if ($cantidad > 0) {
7             $this->saldo += $cantidad;
8         }
9     }
10
11     public function obtenerSaldo() {
12         return $this->saldo;
13     }
14 }
15
16 $cuenta = new Cuenta();
17 $cuenta->depositar(100);
18 $cuenta->depositar(100);
19 echo "Saldo actual: " . $cuenta->obtenerSaldo() . " €";
20 ?>
```

Salida:

Saldo actual: 200 €

Programa 8 - Herencia

La herencia permite que una clase reciba propiedades y métodos de otra. Aquí **Perro** hereda de **Animal** y sobrescribe el método **hacerSonido()**. Dentro de la versión sobrescrita uso **parent::hacerSonido()** para llamar al método original de la clase padre.

Programa8.php · líneas 1-27

```
1 <?php
2 class Animal {
3     public $nombre;
4
5     public function __construct($nombre) {
6         $this->nombre = $nombre;
7     }
8
9     public function hacerSonido() {
10         return "El animal hace un sonido";
11     }
12 }
13
14 // Clase Perro que hereda de Animal
15 class Perro extends Animal {
16     public function hacerSonido() {
17         $padrehabla = parent::hacerSonido();
18         return "Guau Guau";
19     }
20 }
21
22
23 $perro = new Perro("Max");
24 echo $perro->hacerSonido(); // Salida: Guau Guau
25
26
27 ?>
```

Salida:

Programa 9 - Polimorfismo

El polimorfismo permite que distintas clases tengan métodos con el mismo nombre pero comportamientos diferentes. Aquí **Cuadrado** y **Circulo** heredan de **Figura** y cada una implementa su propio **calcularArea()**. Después se recorren con un **foreach** tratándolas como si fueran del mismo tipo.

```
1 <?php
2 class Figura {
3     public function calcularArea() {
4         return 0;
5     }
6 }
7
8 class Cuadrado extends Figura {
9     private $lado;
10
11     public function __construct($lado) {
12         $this->lado = $lado;
13     }
14
15     public function calcularArea() {
16         return $this->lado * $this->lado;
17     }
18 }
19
20 class Circulo extends Figura {
21     private $radio;
22
23     public function __construct($radio) {
24         $this->radio = $radio;
25     }
26
27     public function calcularArea() {
28         return pi() * ($this->radio ** 2);
29     }
30 }
31
32 // Polimorfismo en acción
33 $figuras = [new Cuadrado(4), new Circulo(3)];
34
35 foreach ($figuras as $figura) {
36     echo "Área: " . $figura->calcularArea() . "\n";
37 }
38 ?>
```

Salida:

Programa 10 - Clases abstractas

Una clase **abstracta** no se puede instanciar directamente, sirve solo como plantilla. Obliga a las clases hijas a implementar los métodos marcados como **abstract**. Aquí **Transporte** es abstracta, y **Coche** y **Bicicleta** están obligadas a definir **mover()**.


```
1 <?php
2 abstract class Transporte {
3     protected $velocidad;
4
5     public function __construct($velocidad) {
6         $this->velocidad = $velocidad;
7     }
8
9     // Método abstracto
10    abstract public function mover();
11 }
12
13 class Coche extends Transporte {
14     public function mover() {
15         return "El coche se mueve a $this->velocidad km/h";
16     }
17 }
18
19 class Bicicleta extends Transporte {
20     public function mover() {
21         return "La bicicleta se mueve a $this->velocidad km/h";
22     }
23 }
24
25 $coche = new Coche(120);
26 echo $coche->mover(); // Salida: El coche se mueve a 120 km/h
27
28 $bicicleta = new Bicicleta(20);
29 echo $bicicleta->mover(); // Salida: La bicicleta se mueve a 20 km/h
30 ?>
```

Salida:

Programa 11 - Interfaces

Una **interfaz** es como un contrato: define qué métodos debe tener una clase, pero no cómo funcionan. Después, cualquier clase que implemente esa interfaz está obligada a definir todos los métodos. Aquí **Cuenta** implementa la interfaz **Operaciones**, así que tiene **depositar()** y **retirar()**.

```
1 <?php
2 interface Operaciones {
3     public function depositar($cantidad);
4     public function retirar($cantidad);
5 }
6
7 class Cuenta implements Operaciones {
8     private $saldo = 0;
9
10    public function depositar($cantidad) {
11        $this->saldo += $cantidad;
12    }
13
14    public function retirar($cantidad) {
15        if ($cantidad <= $this->saldo) {
16            $this->saldo -= $cantidad;
17        }
18    }
19
20    public function obtenerSaldo() {
21        return $this->saldo;
22    }
23 }
24
25 $cuenta = new Cuenta();
26 $cuenta->depositar(100);
27 $cuenta->retirar(30);
28 echo $cuenta->obtenerSaldo(); // Salida: 70
29 ?>
```

Salida:

Programa 12 - **final**

La palabra clave **final** impide que una clase se herede o que un método se sobrescriba. Se usa para proteger lógica crítica que no se debe cambiar.

Clase **Calculadora** declarada como **final**:

Programa12.php · líneas 1-21

```
1 <?php
2 // Clase final: no se puede heredar de ella
3 final class Calculadora {
4     public function sumar($a, $b) {
5         return $a + $b;
6     }
7
8     public function restar($a, $b) {
9         return $a - $b;
10    }
11 }
12
13 // Crear un objeto y usar la clase
14 $calc = new Calculadora();
15 echo $calc->sumar(5, 3); // Resultado: 8
16 echo "<br>";
17 echo $calc->restar(10, 4); // Resultado: 6
18
19 // Esto provocaría un error fatal:
20 // class MiCalculadora extends Calculadora {}
21 ?>
```

Método `final` dentro de una clase (`Base::conectar()`):

Programa12.php · líneas 23-37

```
23 <?php //Métodos final
24 class Base {
25     final public function conectar() {
26         echo "Conexión establecida";
27     }
28 }
29
30 class Derivada extends Base {
31     // Error si ... ???
32     // public function conectar() {
33 }
34
35 // Crear un objeto y usar la clase
36
37 ?>
```

Clase `Email` como objeto-valor inmutable y final:

```
39 <?php
40 final class Email
41 {
42     private string $value;
43
44     public function __construct(string $value)
45     {
46         // Validación simple
47         if (!filter_var($value, FILTER_VALIDATE_EMAIL)) {
48             throw new InvalidArgumentException("Email no válido");
49         }
50         $this->value = strtolower($value);
51     }
52
53     public function value(): string
54     {
55         return $this->value;
56     }
57
58     // Comparación de valor (útil en tests)
59     public function equals(Email $other): bool
60     {
61         // Comparar valores en minúsculas para ignorar mayúsculas/minúsculas
62         return $this->value === $other->value;
63     }
64
65     public function __toString(): string
66     {
67         return $this->value;
68     }
69 }
70
71 // Uso
72 $e1 = new Email("Alumno@IES.edu");
73 $e2 = new Email("alumno@ies.edu");
74 var_dump($e1->equals($e2)); // bool(true)
75 echo "<br>";
76 echo "Clase Email usada correctamente., Eliminada la herencia de la clase Email.";
```

Salida:

Programa 13 - Traits

Los **traits** son una forma de compartir métodos entre varias clases sin recurrir a la herencia. Básicamente son un "bloque de métodos" que se pueden inyectar en cualquier clase con **use**. Aquí el trait **Registro** aporta el método **registrarAccion()**, que reutilizan tanto **Usuario** como **Producto**.

Definición del trait:

Programa13.php · líneas 1-8

```
1 <?php
2 // Definición del trait
3 trait Registro {
4     public function registrarAccion($mensaje) {
5         $fecha = date('Y-m-d H:i:s');
6         echo "[$fecha] $mensaje<br>";
7     }
8 }
```

Clases que lo usan:

```
11 // Clase que usa el trait
12 class Usuario {
13     use Registro; // "inyecta" los métodos del trait
14
15
16     public function login($nombre) {
17         $this->registrarAccion("El usuario '$nombre' ha iniciado sesión.");
18     }
19 }
20
21
22 class Producto {
23     use Registro; // "inyecta" los métodos del trait
24
25
26     public function crear($nombre) {
27         $this->registrarAccion("Se ha creado el producto '$nombre'.");
28     }
29 }
30
31
32 // Uso
33 $u = new Usuario();
34 $u->login("francisco.perez");
35
36
37
38 $p = new Producto();
39 $p->crear("Camiseta");
40 ?>
```

Salida:

Programa 14 - Namespaces

Los **namespaces** sirven para organizar el código y evitar que dos clases con el mismo nombre choquen entre sí. Aquí defino las clases **Admin** y **Guest** dentro del namespace **Users**, y desde **Main.php** las importo con **use** y las uso normalmente.

Archivo **Main.php** (importa y usa las clases):

Main.php · líneas 1-18

```
1 <?php
2 require_once 'Users/Admin.php';
3 require_once 'Users/Guest.php';
4
5
6 use Users\Admin;
7 //use Users\Guest;
8
9
10 $admin = new Admin();
11 echo "<br> Rol del usuario: " . $admin->getRole(); // Output: Rol del usuario: Administrador
12
13
14 $guest = new Users\Guest();
15 echo "<br> Rol del usuario: " . $guest->getRole(); // Output: Rol del usuario: Invitado
16
17
18 ?>
```

Archivo `Users/Admin.php`:

`Admin.php` · líneas 1-12

```
1 <?php
2 namespace Users;
3
4
5 class Admin {
6     public function getRole() {
7         return "Administrador";
8     }
9 }
10
11
12 ?>
```

Archivo `Users/Guest.php`:

`Guest.php` · líneas 1-10

```
1 <?php
2 namespace Users;
3
4 class Guest {
5     public function getRole() {
6         return "Invitado";
7     }
8 }
9
10 ?>
```

Salida:

Programa 15 - SplStack (pilas)

SplStack es una clase de la Standard PHP Library (SPL) que implementa una **pila** (estructura LIFO: el último que entra es el primero que sale). Para meter elementos se usa **push()** y para sacarlos **pop()**. Útil para historiales de navegación, "deshacer" en editores, etc.

```
1 <?php
2 // Crear una nueva pila
3 $stack = new SplStack();
4
5
6 // Añadir elementos a la pila
7 $stack->push("Primer elemento");
8 $stack->push("Segundo elemento");
9 $stack->push("Tercer elemento");
10
11
12 // Desapilar elementos (se eliminarán en orden inverso)
13 echo $stack->pop(); // Output: Tercer elemento
14 echo "<br>";
15 echo $stack->pop(); // Output: Segundo elemento
16 echo "<br>";
17 echo $stack->pop(); // Output: Primer elemento
18
19
20 ?>
```

Salida:

Tercer elemento
Segundo elemento
Primer elemento

Programa 15b - SplQueue (colas) y SplFileObject

`SplQueue` es lo contrario de la pila: una **cola** FIFO (el primero que entra es el primero que sale). Se usa `enqueue()` para meter y `dequeue()` para sacar. También aprovecho para probar `SplFileObject`, que es una forma orientada a objetos de leer ficheros línea a línea.

`SplQueue`:

Programa15b.php · líneas 1-16

```
1 <?php
2 // Crear una nueva cola
3 $queue = new SplQueue();
4
5 // Añadir elementos a la cola
6 $queue->enqueue("Primero");
7 $queue->enqueue("Segundo");
8 $queue->enqueue("Tercero");
9
10 // Sacar elementos de la cola (FIFO: First In, First Out)
11 echo $queue->dequeue(); // Output: Primero
12 echo "\n";
13 echo $queue->dequeue(); // Output: Segundo
14 echo "\n";
15 echo $queue->dequeue(); // Output: Tercero
16 ?>
```

`SplFileObject` leyendo el propio `Entrega3.md`:

Programa15b.php · líneas 17-31

```
17 <?php
18 $file = new SplFileObject('Entrega3.md', 'r');
19
20 // iterate over its contents
21 while (!$file->eof()) {
22     // get the current line
23     $line = $file->fgets();
24     echo $line;
25     // trim it, and then check if its empty
26     if (empty(trim($line))) {
27         // skips the current iteration
28         continue;
29     }
30 }
31 ?>
```

Salida:

Primero Segundo Tercero# UD 3 PHP POO ## Teoria Ganbeta <https://www.genbeta.com/inteligencia-artificial/herramientas-programacion-ia-estan-provocando-que-desarrolladores-codifiquen-cada-vez-peor> > * "Copilot supone un increíble ahorro de tiempo... si ya sabes lo que vas a escribir". * Para mí, personalmente, la IA es una herramienta que permite reducir el tiempo, (escribir código o buscar información), lo que antes se necesitaba mucho más tiempo para encontrar una documentación específica. MiniResumen La programación orientada a objetos (en adelante POO) es un paradigma de programación que surge en Noruega en 1967![\[1760612066725\]\(image/Entrega3/1760612066725.png\)](#) Las bases de la POO son abstracción, encapsulamiento, herencia y polimorfismo. A -Ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación 1. Mantenibilidad 2. Reutilización de Código 3. Escalabilidad 4. Mejora en la Prueba y Depuración 5. Facilita el Trabajo en Equipo 6. Flexibilidad en la Presentación 7. Facilidad para Implementar APIs y Servicios Web 8. Uso de Patrones de Diseño y Buenas Prácticas Ejemplos practicos <https://www.linkedin.com/pulse/ejemplos-pr%C3%A1cticos-de-programaci%C3%B3n-orientada-objetos-ezequiel-xzjh/> ## Programa 1 Creacion de clases, en ella hemos unido una variable numero que tiene como valor 10 y hemos creado una funcion para mostrar este valor con la cadena elegida [!\[1760613326271\]\(image/Entrega3/1760613326271.png\)](#) [!\[1760613275925\]\(image/Entrega3/1760613275925.png\)](#) Para llamar a las funciones de una clase primero nos crearemos un objeto de esa clase y llamaremos a las funciones con `***->***` [!\[1760613393737\]\(image/Entrega3/1760613393737.png\)](#) --- ## Programa 2 Declaramos instancias de la clase Animal, es un poco ampliacion del anterior solo que he cambiado el nombre de la class creada y añadido variables nuevas [!\[1760613718120\]\(image/Entrega3/1760613718120.png\)](#) [!\[1760613845160\]\(image/Entrega3/1760613845160.png\)](#) Segunda parte consiste en arreglar el programa dado, habria que añadir el \$marca y llamar a las funciones de manera correcta [!\[1760614043598\]\(image/Entrega3/1760614043598.png\)](#) [!\[1760614082398\]\(image/Entrega3/1760614082398.png\)](#) --- ## Programa 3 Crear varias instancias de Persona, el `__destruct` se ejecuta siempre al final del codigo, si lo tienes declarado se aplica el `__destruct`, no hace falta llamar a la funcion. [!\[1760614591747\]\(image/Entrega3/1760614591747.png\)](#) [!\[1760614600377\]\(image/Entrega3/1760614600377.png\)](#) --- ## Programa 4 Explicaciond de como realizar getter y setter en una Clase, en este caso con numeros no olvidar que `*` que puede devolver null! [!\[1760615425105\]\(image/Entrega3/1760615425105.png\)](#) [!\[1760615445123\]\(image/Entrega3/1760615445123.png\)](#) --- ## Programa 5 Explicacion de this, en ela Clase C al ser una función estatica podemos verlo en la linea 43 y 44, no hace falta hacer una carga de la instancia para invocar la funcion [!\[1760701607521\]\(image/Entrega3/1760701607521.png\)](#) [!\[1760701562931\]\(image/Entrega3/1760701562931.png\)](#) --- ## Programa 6 Las propiedades y métodos pueden compartir nombre porque viven en espacios separados, y el contexto (acceso a valor vs. llamada a función) determina cuál se usa, en este caso PHP_EOL es un '\n' el resultado cambia en funcion de como invoquemos al codigo en este caso con o sin parentesis [!\[1760703113866\]\(image/Entrega3/1760703113866.png\)](#) [!\[1760702172496\]\(image/Entrega3/1760702172496.png\)](#) --- ## Programa 7 He llamado a la fucion varias veces para incrementar su saldo, con este programa se prueba el encapsulamiento, no podiamos acceder a la variable \$saldo directamente por ser de tipo private si usamos el dolarthis no necesitaremos poner dollar a la variable invocada [!\[1760703629089\]\(image/Entrega3/1760703629089.png\)](#) ! [\[1760702200370\]\(image/Entrega3/1760702200370.png\)](#) --- ## Programa 8 La herencia permite que una clase herede propiedades y métodos de otra clase, promoviendo la reutilización de código. Clase padre: Es la clase de la cual se hereda. Clase hija: Es la clase que hereda de la clase padre. [!\[1760704151567\]\(image/Entrega3/1760704151567.png\)](#) [!\[1760704341133\]\(image/Entrega3/1760704341133.png\)](#) --- ## Programa 9 El `**polimorfismo**` significa que diferentes clases pueden tener métodos con el mismo nombre, pero cada uno hace cosas diferentes, permitiendo usar todas las clases de manera similar. [!\[1760704494883\]\(image/Entrega3/1760704494883.png\)](#) [!\[1760702272361\]\(image/Entrega3/1760702272361.png\)](#) --- ## Programa 10 La abstracción crea una clase que no se puede usar directamente, pero sirve de modelo para que otras clases (hijas) definan su propio comportamiento, usando métodos que la clase base exige que implementen. [!\[1760705208551\]\(image/Entrega3/1760705208551.png\)](#) [!\[1760702237172\]\(image/Entrega3/1760702237172.png\)](#) --- ## Programa 11 Explicacion de las Interfaces Una interfaz es un contrato que dice qué métodos debe tener una clase, pero no dice cómo funcionan; las clases que la usan deben implementar todos esos métodos. [!\[1760705411693\]\(image/Entrega3/1760705411693.png\)](#) --- ## Programa 12 final se usa para evitar que una clase o método sea heredado o sobrescrito. En Calculadora y Email, declarar la clase como final impide crear subclases, por eso `class MiCalculadora extends Calculadora {}` da error. En Base, el método conectar() es final, lo que significa que Derivada no puede redefinirlo, y cualquier intento provocaría error. Usar final protege lógica crítica o estable, evitando que otros cambien su comportamiento por herencia. Fallan cuando se intenta heredar de una clase final o sobrescribir un método final, porque PHP lo considera un error fatal. [!\[1760960014413\]\(image/Entrega3/1760960014413.png\)](#) [!\[1760959530741\]\(image/Entrega3/1760959530741.png\)](#) --- ## Programa 13 Este ejemplo muestra cómo usar traits en PHP para compartir funcionalidad entre distintas clases sin necesidad de herencia. Un trait es básicamente un conjunto de métodos que se puede "inyectar" en cualquier clase, permitiendo reutilizar código de manera sencilla. En este caso, el trait Registro añade la capacidad de registrar acciones con fecha y hora, y tanto la clase Usuario como la clase Producto lo utilizan para registrar eventos específicos, como iniciar sesión o crear un producto. Gracias a los traits, podemos agregar comportamiento común sin duplicar código, haciendo que el proyecto sea más limpio y fácil de mantener. Este patrón demuestra cómo la composición de comportamiento es útil para organizar y reutilizar funcionalidades en PHP. [!\[1760960626651\]\(image/Entrega3/1760960626651.png\)](#) [!\[1760960604242\]\(image/Entrega3/1760960604242.png\)](#) --- ## Programa 14 Este ejemplo muestra cómo utilizar namespaces en PHP para organizar clases y evitar conflictos de nombres. Se definen dos clases, Admin y Guest, dentro del namespace Users, cada una con un método getRole() que devuelve su rol correspondiente. Al usar `require_once` y la declaración use, podemos importar y crear objetos de estas clases de manera clara y sin ambigüedad. Esto permite trabajar con múltiples clases que podrían tener el mismo nombre en diferentes partes del proyecto sin generar errores. En general, los namespaces ayudan a estructurar mejor el código y mantenerlo ordenado en proyectos grandes. [!\[1760962158444\]\(image/Entrega3/1760962158444.png\)](#) [!\[1760961743489\]\(image/Entrega3/1760961743489.png\)](#) [!\[1760961765011\]\(image/Entrega3/1760961765011.png\)](#) [!\[1760961803984\]\(image/Entrega3/1760961803984.png\)](#) --- ## Programa 15 Este ejemplo muestra cómo usar la clase SplStack de PHP para trabajar con pilas (estructuras LIFO: Last In, First Out). Se crean elementos y se agregan a la pila usando push(), mientras que pop() permite desapilar los elementos en orden inverso al que fueron añadidos. Esto ilustra cómo manejar colecciones donde el último elemento agregado es el primero en ser procesado. La clase SplStack proporciona una manera eficiente y lista para usar de implementar este tipo de estructura de datos en PHP. En general, es útil para casos como historial de acciones, navegación o cualquier proceso que requiera orden inverso de ejecución. [!\[1760962784975\]\(image/Entrega3/1760962784975.png\)](#) [!\[1760962726434\]\(image/Entrega3/1760962726434.png\)](#) --- ## Programa 15 b Este ejemplo muestra cómo usar la clase SplQueue de PHP para trabajar con colas (estructuras FIFO: First In, First Out). Se añaden elementos a la cola con enqueue() y se retiran con dequeue(), lo que permite procesarlos en el orden en que fueron agregados. Esto ilustra cómo manejar colecciones donde el primer elemento en entrar es el primero en salir, como en colas de impresión o tareas pendientes. La clase SplQueue proporciona una manera sencilla y eficiente de implementar este comportamiento sin tener que programarlo desde cero. En general, es útil para organizar procesos que requieren un flujo de ejecución ordenado según su llegada. Artículo: <https://blogprog.gonzalolopez.es/articulos/biblioteca-spl-de-php.html> [!\[1760963048782\]\(image/Entrega3/1760963048782.png\)](#) [!\[1760963014885\]\(image/Entrega3/1760963014885.png\)](#) --- ## Programa 16 [!\[1760964550412\]\(image/Entrega3/1760964550412.png\)](#) [!\[1760964488135\]\(image/Entrega3/1760964488135.png\)](#) [!\[1760964508123\]\(image/Entrega3/1760964508123.png\)](#) [!\[1760964534575\]\(image/Entrega3/1760964534575.png\)](#)

Programa 16 - Autoload con spl_autoload_register

Cuando un proyecto tiene muchas clases, escribir un `require_once` por cada archivo se hace pesadísimo. La solución es **autoload**: una función que se registra con `spl_autoload_register` y que PHP llama automáticamente cuando intentas usar una clase que no está cargada.

`index.php` con el autoload:

index.php · líneas 1-10

```
1 <?php
2 spl_autoload_register(function ($class_name) {
3     include 'classes/' . $class_name . '.php';
4 });
5
6 $u = new User();
7 $p = new Product();
8
9
10 ?>
```

classes/User.php:

User.php · líneas 1-10

```
1 <?php
2 class User {
3     public function __construct() {
4         echo "Clase User cargada.\n";
5     }
6 }
7
8
9 ?>
10
```

classes/Product.php:

Product.php · líneas 1-7

```
1 <?php
2 class Product {
3     public function __construct() {
4         echo "Clase Product cargada.\n";
5     }
6 }
7 ?>
```

Salida:

Conclusiones

Lo que me llevo de esta unidad:

- POO **no es solo "meter clases por meterlas"**, sino que tiene unos pilares concretos (encapsulamiento, herencia, polimorfismo, abstracción) que ayudan a estructurar mejor el código.
- Saber cuándo usar herencia y cuándo **trait** o **interface** es clave: la herencia es para clases que son "del mismo tipo", los traits para compartir comportamiento entre clases distintas, y las interfaces para garantizar que algo cumple un contrato.
- Las clases **abstract** y **final** son herramientas para forzar o impedir cosas según convenga.
- Los **namespaces** y el **autoload** son básicos en cualquier proyecto medianamente grande.
- La **SPL** ya trae estructuras de datos hechas (pilas, colas, ficheros...), así que no hace falta reinventarlas.

Esta unidad es una de las más densas porque cambia la forma de pensar el código, pero también es donde más se nota la diferencia con respecto al PHP que veníamos haciendo.